
SmartShield: An Intelligent AI-Based Cybersecurity Platform for Integrated Threat Detection

By

Md Sabbir Hossain	21223203078
Bipro Saha	20211203049
Afrin Jenan Lita	21223203084
Md. Nazmul Huqe	21223203087
Md. Atikul Islam	21223203057

Submitted in partial fulfilment of the requirements of the degree of
Bachelor of Science and Engineering in
Computer Science and Engineering



Department of Computer Science and Engineering
Bangladesh University of Business and Technology

July 2026

Declaration

We do hereby declare that the research works presented in this report entitled, **SmartShield: An Intelligent AI-Based Cybersecurity Platform for Integrated Threat Detection** are the results of our works. We further declare that the report has been compiled and written by us, and no part of this report has been submitted elsewhere for the requirements of any degree, award, diploma, or any other purposes except for publications. The materials that are obtained from other sources are duly acknowledged in this report.

Md. Sabbir Hossain
21223203078

Md. Nazmul Huque
21223203087

Bipro Saha
20211203049

Md. Atikul Islam
21223203057

Afrin Jenan Lita
21223203084

Approval

I do hereby declare that the research works presented in this report entitled, **SmartShield: An Intelligent AI-Based Cybersecurity Platform for Integrated Threat Detection**, are the outcome of the original works carried out by the Md. Sabbir Hossain, Bipro Saha, Afrin Jenan Lita, Md. Nazmul Hujje, Md. Atikul Islam under my supervision. I further declare that no part of this report has been submitted elsewhere for the requirements of any degree, award, diploma, or other purposes except for publications. I further certify that the dissertation meets the requirements and standards for the degree of Bachelor of Science in Computer Science and Engineering.

Supervisor

Md. Masudul Islam
Assistant professor
Department of Computer Science & Engineering
Bangladesh University of Business and Technology
Dhaka, Bangladesh

Chairman

Prof. Dr. Md. Ahsan Habib
Professor & Chairman
Department of Computer Science & Engineering
Bangladesh University of Business and Technology
Dhaka, Bangladesh

Dedication

We would like to dedicate this research to our loving parents.

Acknowledgment

We would like to express our heartfelt gratitude to the almighty Allah who offered our family and us kind care throughout this journey. We are deeply thankful to the Bangladesh University of Business and Technology (BUBT) for providing us with such a wonderful environment to pursue our project. We would like to express our sincere gratitude to **Md. Masudul Islam**, Assistant Professor, Department of CSE, BUBT. We have completed our research with his help. We found the research area, topic, and problem with his suggestions. He guided us with our study and supplied us with many research papers and academic resources in this area. He is patient and responsible. When we had questions and needed his help, he would always find time to meet and discuss with us no matter how busy he was. We would also like to acknowledge our team members for supporting each other and be grateful to our university for providing this opportunity for us.

Abstract

The rapid growth of cyber threats has created significant challenges for traditional security systems, which often struggle to detect sophisticated and emerging attacks. Among the most critical cybersecurity threats are Intrusion Detection System (IDS) attacks, malware, and phishing emails, all of which can lead to severe financial loss, data breaches, and operational disruption. To address these challenges, this research proposes SmartShield, an AI-driven unified cybersecurity ecosystem that integrates intelligent detection of these three major cyber threats into a single platform. Three publicly available benchmark datasets were selected for intrusion detection, malware classification, and phishing email detection. Each dataset was independently preprocessed, and multiple machine learning algorithms were trained and evaluated using standard performance metrics, including Accuracy, Precision, Recall, and F1-score, to identify the most suitable model for deployment. Experimental results show that the Optimized XGBoost model achieved the highest performance for the IDS dataset with an accuracy of 95.96%, while Random Forest obtained the best accuracy of 92.71% for malware classification. For phishing and spam email detection, the Support Vector Machine (SVM) outperformed other classifiers by achieving an accuracy of 99.67%. Based on these findings, the best-performing models from each module were integrated into the proposed SmartShield system. The developed application provides a centralized platform capable of analyzing network traffic, classifying malware samples, and detecting phishing or spam emails through an intelligent email classification module. Prediction results are displayed in real time through a unified dashboard, enabling faster threat identification and response. The proposed SmartShield framework demonstrates that integrating task-specific machine learning models into a unified architecture can significantly enhance cybersecurity performance while providing a scalable, practical, and efficient solution for protecting modern digital environments.

List of Figures

No.	Caption	Page No.
3.1	Our proposed system architecture and AI deployment	8
3.2	Applied ML method for Intrusion Detection System	12
3.3	Applied ML method for Malware Classification Model	12
3.4	Applied ML method of Phishing Detection Model	13
4.1	Confusion matrix for Malware Detection	22
4.2	Confusion matrix for Phishing Detection	23
4.3	Confusion matrix for Intrusion Detection	24
4.4	Dashboard of the Intrusion Detection Module	27
4.5	Dashboard of Phishing Detection Module	27
4.6	Dashboard of the Malware Classification Module	28
4.7	Dashboard of the event history	28
4.8	Security alert in dashboard	29
5.1	Timeline and Gantt Chart	34

List of Tables

No.	Caption	Page No.
2.1	Summary overview of literature review	6-7
3.1	Summary of CICIDS2018 dataset	9
3.2	Summary of Microsoft Malware classification challenge (BIG 2015) dataset	9-10
3.3	Summary of Phishing email dataset	10
3.4	Hyperparameter settings summary	11
4.1	Environment Setup of the proposed method	16
4.2	Result comparison for malware classification	21
4.3	Result comparison for Spam and phishing detection	22
4.4	Result comparison for Intrusion Detection	23

List of Abbreviations and Acronyms

AI	Artificial Intelligence
ML	Machine Learning
CNN	Convolutional Neural Network
DNN	Deep Neural Network
GNN	Graph Neural Network
IDS	Intrusion Detection System
DDOS	Distributed Denial of Service
GraphSAGE	Graph Sample and Aggregate
CIC-IDS	Canadian Institute for Cybersecurity Intrusion Detection System 2017 Dataset
CIC-MalMem	Canadian Institute for Cybersecurity Malware Memory Dump Dataset 2022
Ember	Endgame Malware Benchmark for Research Dataset
SVM	Support Vector Machine
LAN	Local Area Network
PE	Portable Executable
REST	Representational State Transfer
RF	Random Forest
SMOTE	Synthetic Minority Over-sampling Technique
LightGBM	Light Gradient Boosting Machine
LSTM	Long Short-Term Memory
NLP	Natural Language Processing
TF-IDF	Term Frequency–Inverse Document Frequency
PCA	Principal Component Analysis
UCI	University of California, Irvine Machine Learning Repository
XGBoost	Extreme Gradient Boosting

Table of Contents

Declaration	ii
Approval	iii
Dedication	iv
Acknowledgment	v
Abstract	vi
List of Figures	vii
List of Tables	viii
List of Abbreviations and Acronyms	ix
1. Introduction	1-4
1.1. Problem Statement	1
1.2. Problem Background	1
1.3. Research Objectives	1
1.4. Motivations	2
1.5. Significance of the Research	2
1.6. Research Contribution	2
1.7. Complex Engineering Analysis	3
1.7.1 Complex Engineering Problems	3
1.7.2 Complex Engineering Activities	3
1.8. Report Organization	4
2. Background Study	5-7
2.1. Literature Review	5
2.2. Problem Analysis	7
3. Research Methodology	8-14
3.1. Proposed Framework	8
3.2. Datasets Analysis	9
3.2.1. Data Collection & Preprocessing	10
3.2.2. Feature Selection	10
3.3. Model Analysis	10
3.3.1. Hyperparameters Setting	10
3.3.2. IDS Detection Model Analysis	11
3.3.3. Malware Classification Model Analysis	12
3.3.4. Spam and Phishing Detection Analysis	13
3.3.5. Proposed Model	13
3.4. Explanation of Different Modules	15
4. Implementation and Result Analysis	16-29
4.1. Environment Setup	16
4.2. Model Implementation	17

4.2.1. Malware Classification Module	17
4.2.2. Email Spam and Phishing Detection Module	18
4.2.3. Intrusion Detection Module	18
4.2.4. Model Deployment	19
4.3. Evaluation Metrics	19
4.3.1. Confusion Matrix	19
4.3.2. Accuracy	20
4.3.3. Precision	20
4.3.4. Recall	20
4.3.5. F1-Score	20
4.3.6 Model Comparison	20
4.4. Performance Analysis	21
4.4.1. Malware Classification Performance	21
4.4.2. Email Spam and Phishing Detection Performance	22
4.4.3. Intrusion Detection System Performance	23
4.4.4. Comparative Analysis	24
4.4.5. Development Model Selection	25
4.5. Key Findings and Discussions	25
4.6. Deployment	26
5 Standards, Constraints, and Milestones	30-33
5.1. Sustainability Standards	30
5.2. Impacts on Society	31
5.3. Ethics	31
5.4. Challenges	32
5.5. Constraints	32
5.5.1. Design Constraints	32
5.5.2. Component Constraints	33
5.5.3. Budget Constraints	33
5.6. Timeline and Gantt Chart	33
6. Conclusion	35-36
6.1 Limitations	35
6.2 Future Works and Direction	36
References	37-38

Chapter 1

1. Introduction

In today's rapidly evolving digital environment, cybersecurity threats have become more complex and sophisticated. Traditional security systems that rely on static rules and fixed signatures often fail to detect new and advanced cyberattacks. As a result, organizations face growing risks of data breaches, ransomware attacks, and financial loss. To address these challenges, artificial intelligence (AI) and machine learning (ML) have emerged as powerful tools that can enhance the accuracy and responsiveness of modern intrusion detection systems (IDS). This research aims to design an AI-based IDS capable of real-time detection, classification, and alerting of cyber threats.

1.1. Problem Statement

Most legacy cybersecurity systems depend on pre-defined rules or known attack signatures. While effective against known threats, these systems struggle to detect novel or zero-day attacks.[1][3] As cybercriminals adopt increasingly advanced strategies such as phishing, ransomware, and denial-of-service (DoS) attacks, the gap between detection and defense continues to widen. The key problem, therefore, lies in the inability of existing systems to adapt and respond dynamically to emerging threats in real time.[3][4]

1.2. Problem Background

Cyberattacks have become more intelligent, automated, and widespread across industries. The lack of adaptability in traditional systems often results in delayed detection, incorrect classification, and inadequate response. This delay leads to severe consequences such as data theft, business downtime, and financial loss. In contrast, AI-based intrusion detection systems can continuously learn from network patterns, identify anomalies, and provide proactive defense mechanisms. Hence, the integration of AI and ML into network security frameworks has become a crucial research area. [1][2]

1.3. Research Objectives

The main goal of this research is to design and develop an integrated cybersecurity system, Smart Shield, that can detect multiple types of cyber threats using machine learning. The specific objectives of this research are as follows:

1. To collect and preprocess suitable datasets for three different types of cyber threats: network intrusion, malware, and phishing/spam emails.

2. To train and evaluate multiple machine learning models for each module and select the best-performing model based on accuracy and other performance metrics.
3. To design an Intrusion Detection module that can capture live network packets from a LAN interface and classify them as normal or malicious in real time.
4. To design a Malware Classification module that can take a file as input, convert it into a suitable vector representation, and classify it into the correct malware family.
5. To design a Phishing and Spam Detection module that can analyze the sender, subject, and body of an email using TF-IDF feature extraction and determine whether the email is legitimate or malicious.
6. To integrate all three detection modules into a single unified system rather than keeping them as separate, disconnected tools.
7. To develop a simple and user-friendly web dashboard where users can monitor all three modules, view detection results, and receive alerts in one place.
8. To evaluate the overall performance and practicality of the proposed system as a low-cost security solution suitable for small to medium-scale networks.

1.4. Motivations

The increasing number of cyber-attacks and their devastating impact on organizations highlight the need for more intelligent and adaptive defense systems.[1][2] The motivation behind this research stems from the limitations of static IDS frameworks and the potential of AI-based systems to offer self-learning, data-driven, and real-time protection. By integrating machine learning and automation, organizations can significantly reduce response time and enhance their overall cyber security posture.

1.5. Significance of the Research

This research contributes to the field of cyber security by introducing an adaptive, AI-powered intrusion detection system capable of recognizing and mitigating both known and novel threats. It aims to minimize false positives, ensure early detection, and support administrative decision-making through an intuitive dashboard. The implementation of such systems can strengthen data protection, improve operational efficiency, and contribute to national and organizational cyber resilience.

1.6. Research Contribution

The project's contribution lies in the development of an integrated AI framework for intrusion detection that combines supervised and unsupervised learning algorithms. It proposes a modular architecture that includes data collection, feature extraction, model training, and real-time detection. Additionally, the project offers a visualization dashboard and an automated alerting system to assist network administrators in monitoring security events efficiently.

1.7. Complex Engineering Analysis

1.7.1. Complex Engineering Problems

P1 – Deep Technical Knowledge Required: This problem requires strong understanding of machine learning, computer networking, and cybersecurity concepts. The engineer must be able to interpret complex traffic behavior and security patterns accurately.

P2 – Speed vs. Accuracy Optimization: The system needs to detect threats quickly while maintaining high accuracy. Achieving real-time performance without increasing false alarms is a key challenge.

P3 – Detection of Unknown Threats: Network traffic must be analyzed at a deeper behavioral level to identify zero-day and previously unseen attacks. This requires intelligent models that learn patterns rather than relying only on signatures.

P4 – Variability of Cyber Attacks: Some attacks are common and well-understood, while others are highly sophisticated and hidden. The system must be adaptive and capable of handling both straightforward and advanced threats.

P5 – Privacy and Legal Compliance: Monitoring network traffic must not violate user privacy or data protection regulations. The system needs to follow cybersecurity laws and organizational security policies.

P6 – Multi-Disciplinary Collaboration: Project success requires teamwork among administrators, security analysts, engineers, and management. Clear communication and coordinated decision-making are essential.

P7 – System Integration Complexity: Multiple modules—traffic capture, ML processing, logging, and dashboard—must operate in sync. Reliable integration ensures smooth detection and response without system failure.

1.7.2. Complex Engineering Activities

A1: The project involves handling large datasets, cloud services, and advanced ML frameworks. It demands professional-level engineering tools and computational infrastructure.

A2: System modules need to interact **in real-time** to detect and respond to threats instantly. This requires efficient communication pipelines and low-latency architecture.

A3: Integrating AI introduces intelligent defense strategies beyond traditional IDS methods. It enhances adaptability and strengthens proactive security posture.

A4: The system contributes to society by reducing cyber-attack risks and protecting data integrity. It supports safer digital operations for organizations and users.

A5: The solution builds upon traditional intrusion detection systems and extends them with AI-driven enhancements. Thus, it improves accuracy, reduces false alarms, and increases threat visibility.

1.8. Report Organization

The remainder of this report is structured as follows: Chapter 1: Introduction, including objectives, motivations, and contributions. Chapter 2: Literature review and problem analysis. Chapter 3: System architecture and methodology. Chapter 4: Implementation and integration. Chapter 5: Experimental results and evaluation. Chapter 6: Conclusion and future directions.

Chapter 2

2. Background Study

This chapter provides a comprehensive understanding of existing research and technologies related to AI-based intrusion detection systems. It highlights previous works, their methodologies, and the limitations that this study aims to overcome.

2.1. Literature Review

Recent advancements in Artificial Intelligence (AI) and Machine Learning (ML) have significantly transformed the field of cybersecurity. Between 2019 and 2025, numerous researchers proposed intelligent approaches to improve the detection of cyber threats such as intrusion attacks, malware, and phishing. These studies demonstrate that AI-based techniques provide higher detection accuracy, better adaptability to emerging threats, and reduced false alarm rates compared to traditional rule-based security systems. To understand the current state of research and identify suitable approaches for our proposed system, we reviewed several recent and influential research papers related to AI-driven cybersecurity. The following summaries highlight the key contributions of six selected studies.

Zhiwei Xu et al. (2025) presented a comprehensive survey on deep learning-based Intrusion Detection Systems (IDS). The study reviewed various datasets and advanced models, including CNN, LSTM, GNN, and Transformer-based architectures. Most of the reviewed methods achieved over 90% detection accuracy while demonstrating strong capability in identifying zero-day attacks and complex network intrusions.[3]

Tony Quertier et al. (2024) proposed a lightweight Transformer-based model for malware detection using API call sequences. The model achieved an accuracy of 99% and an F1-score of 0.99 on the B&PEML dataset while maintaining low computational complexity. Their work demonstrated that efficient AI models can provide high malware detection performance with reduced resource consumption.[4]

Hadia Bibi et al. (2024) developed a multilayer CNN model for phishing website detection using the UCI Phishing Website Dataset. Their proposed approach outperformed several traditional machine learning algorithms and achieved 99.1% accuracy, together with high precision, recall, and F1-score. The research highlighted the effectiveness of deep learning for detecting phishing attacks.[5]

Wilk-Jakubowski et al. (2025) conducted a systematic review of machine learning and neural network techniques for phishing detection. The study analyzed multiple public datasets and compared algorithms such as Random Forest, SVM, XGBoost, CNN, and LSTM. The review

reported that most modern AI-based phishing detection models achieved 93% to over 99% accuracy, emphasizing the growing effectiveness of intelligent detection techniques.[6]

S. M. Kasongo and Y. Sun (2019) introduced a deep neural network with filter-based feature engineering for wireless intrusion detection. Their proposed framework achieved approximately 99% detection accuracy while reducing computational complexity through effective feature selection. The study demonstrated that proper feature engineering can significantly improve IDS performance.[7]

S. Berman et al. (2019) presented a comprehensive survey on deep learning applications in cybersecurity, covering malware classification, intrusion detection, spam filtering, and botnet detection. The review reported that deep learning models such as CNN, RNN, LSTM, and Autoencoders generally achieved 95–99% detection accuracy across various cybersecurity applications, highlighting the growing importance of AI in modern cyber defense systems.[8]

Table 2.1 shows the literature review summary.

Table 2.1 Summary overview of literature review

Study	Dataset	Model	Result	Remark
Zhiwei Xu et al. (2025) [3]	NSL-KDD, CICIDS2017, UNSW-NB15 and other public IDS datasets	Deep Learning-based IDS (LSTM, GNN, Transformer, GraphSAGE, GAT, CNN, Autoencoder, etc.)	Many DL-IDS approaches reported over 90% detection accuracy and improved zero-day attack detection.	Comprehensive survey of DL-based IDS covering data collection, storage, parsing, detection, and investigation.
Tony Quertier et al. (2024) [4]	Quo Vadis, B&PEML, OB MID-24	Encoder-Only Transformer (Small Language Model)	Quo Vadis: F1=0.88, Accuracy=0.88; B&PEML: F1=0.99, Accuracy=0.99	Lightweight Transformer-based malware detector using API-call sequences with reduced computational cost.
Hadia Bibi et al. (2024) [5]	UCI Phishing Websites Dataset (11,055 websites; 4,898 phishing, 6,157 legitimate)	Decision Tree, Naive Bayes, SVM, Random Forest, Multilayer CNN	Multilayer CNN achieved 99.1% accuracy, 97% precision, 96% recall, and 96% F1-score.	The proposed multilayer CNN outperformed traditional ML models and effectively detected phishing websites using 30 selected features.
Wilk-Jakubowski et al. (2025)[6] [4]	Systematic review of phishing datasets from UCI, PhishTank, OpenPhish, Kaggle, Alexa,	Review of ML and NN approaches (RF, SVM, XGBoost, CNN, LSTM, DNN, ensemble methods, etc.)	Reported accuracies across reviewed studies generally ranged from ~93% to >99%, with CNN/LSTM and ensemble models frequently achieving	Provides a comprehensive comparison of phishing detection techniques, datasets, validation practices, data leakage risks, and future research directions.

Study	Dataset	Model	Result	Remark
	Mendeley, ISCX and others.		the highest performance.	
S. M. Kasongo and Y. Sun (2019) [7]	AWID Wireless Intrusion Detection Dataset	Deep Neural Network (DNN) with Filter-Based Feature Engineering	Achieved approximately 99% detection accuracy while reducing computational complexity.	Demonstrated that feature engineering significantly improves intrusion detection performance and reduces redundant features before classification.
S. Berman et al. (2019) [8]	Multiple cybersecurity datasets (Malware, IDS, Spam/Phishing, Botnet, DDoS)	Deep Learning Models (CNN, RNN, DNN, Autoencoder, LSTM, etc.)	Reported that deep learning models generally achieved 95%–99% detection accuracy across different cybersecurity applications.	Comprehensive survey highlighting the advantages of deep learning for Malware Classification, intrusion detection, spam filtering, and other cybersecurity tasks while discussing future research directions.

2.2. Problem Analysis

Existing IDS solutions primarily depend on signature-based detection methods, making them ineffective against emerging or zero-day attacks.[9] The evolving complexity of network traffic requires systems that can learn from patterns and detect deviations in real time. The proposed system addresses these gaps by combining both supervised learning (for known attack classification) and unsupervised learning (for anomaly detection), ensuring comprehensive and adaptive threat analysis.

Chapter 3

3. Research Methodology

The proposed system collects raw network and malware-related datasets and applies data preprocessing to extract meaningful features. These features are used to train machine learning models for classification of normal and malicious behavior. In the real-time environment, the server captures live network traffic from the LAN interface, and the trained AI model inspects the incoming data. If suspicious activity is identified, the system stores the detection record in the log database, and the results are displayed on a dashboard through an API-based communication layer. This process enables continuous and automated monitoring of cyber threats.

3.1. Proposed System Architecture

This diagram represents how the system works in real-time. When a network attack occurs, the traffic first passes through the LAN card of the monitoring server. The server captures the traffic and forwards it to the AI-based detection module, where the trained model analyses the behavior of the network packets. **Figure 3.1** represents the overall system architecture of our proposed system.

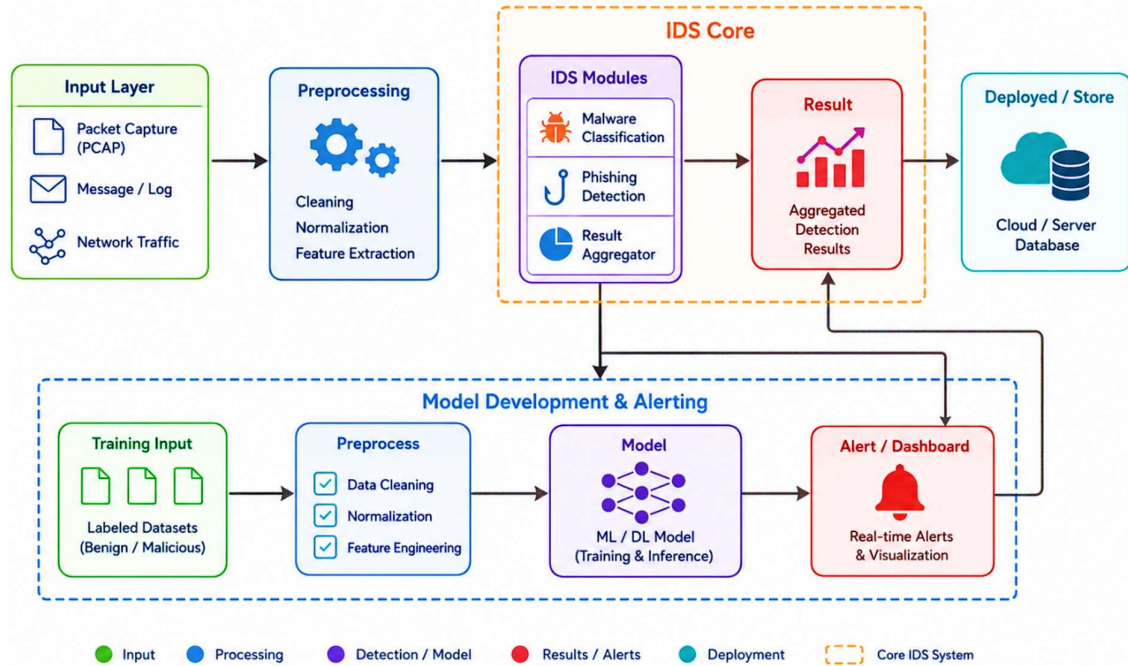


Figure 3.1. Our proposed system architecture and AI deployment.

If the model detects any suspicious or malicious activity, it records the incident into the detection log database. An API service then retrieves the detection results and sends them to a Monitoring Dashboard, where the administrator can observe alerts and take appropriate action.

3.2. Datasets Analysis

To build the three modules of Smart Shield, we used three separate datasets, one for each type of cyber threat. Each dataset was first cleaned and preprocessed, and then split into training and testing sets using an 80:20 ratio, meaning 80% of the data was used to train the models and the remaining 20% was used to test their performance.

For the Intrusion Detection module, we used the CICIDS2018 dataset [10], which contains a total of 175,341 records. The data is divided into two classes: label (0) represents normal/benign traffic, and label (1) represents malicious/attack traffic. **Table 3.1** shows the number of samples in each class along with the training and testing split.

Table 3.1. Summary of CICIDS2018 dataset (IDS)

Classes	Number of data	Train data (80%)	Test data (20%)
label (0)	56000	44800	11200
label (1)	119341	95473	23868
Total	175341	140273	35068

For the Malware Classification module, we used the Vectorized byte-file dataset sampled from Microsoft Malware Classification Challenge (BIG 2015), which contains a total of 1,642 samples belonging to 9 different malware families, namely Ramnit, Lollipop, Kelihos_ver3, Vundo, Simda, Tracur, Kelihos_ver1, Obfuscator.ACY, and Gatak. Most classes contain 200 samples each, except for the Simda family, which contains only 42 samples.[11] **Table 3.2** shows the class-wise distribution of the dataset along with the training and testing split.

Table 3.2. Summary of microsoft malware classification challenge (BIG 2015) dataset

Classes	Malware Family	Number of data	Train (80%)	Test (20%)
1	Ramnit	200	160	40
2	Lollipop	200	160	40
3	Kelihos_ver3	200	160	40
4	Vundo	200	160	40
5	Simda	42	33	9

6	Tracur	200	160	40
7	Kelihos_ver1	200	160	40
8	Obfuscator.ACY	200	160	40
9	Gatak	200	160	40
Total	9	1642	1,313	329

For the Phishing and Spam Detection module, we used a combined dataset built from Enron, Ling, CEAS, Nazario, Nigerian Fraud, and SpamAssassin sources, containing a total of 39,154 email samples.[12] The data is divided into two classes: Class (0) represents legitimate emails, and Class (1) represents phishing/spam emails. **Table 3.3** shows the class-wise distribution of the dataset along with the training and testing split.

Table 3.3 Summary of phishing email Dataset

Classes	Number of data	Train data (80%)	Test Data (20%)
Class (0)	17312	13850	3462
Class (1)	21842	17473	4369
Total	39154	31324	7831

3.2.1. Data Collection & Preprocessing

Data were collected from both standard datasets and real-time network traffic using tools like Zeek. Preprocessing involved removing redundant entries, handling missing values, and normalizing data for consistency. Important network features such as protocol type, packet size, IP flags, and time intervals were extracted using Python-based scripts to prepare the dataset for machine learning models.

3.2.2. Feature Selection

Feature selection was performed using correlation analysis and mutual information to identify the most significant attributes contributing to threat detection. Redundant or irrelevant features were removed to enhance the model’s accuracy and reduce computational cost.[13]

3.3. Model Analysis

3.3.1 Hyperparameters Settings

Hyperparameter tuning plays an important role in improving the performance and generalization capability of machine learning models. Before training, several hyperparameters were carefully selected and optimized through experimental evaluation to achieve the best classification performance for each detection module. Different models required different parameter configurations depending on the characteristics of their respective datasets. The

selected hyperparameters include learning rate, maximum tree depth, number of estimators, batch size, and the number of training epochs, where applicable. These settings were chosen to ensure a balance between model accuracy, computational efficiency, and the ability to generalize to unseen data [14]. **Table 3.4** summarizes the final hyperparameter settings used for the IDS, Malware Classification, and Phishing Detection models in the proposed SmartShield system.

Table 3.4. Hyperparameter Settings Summary

Category	Parameters	Values
IDS	Tuned using Optuna	(15 trials, maximize accuracy)
	Best trial achieved	95.96%
	N_estimators	274
	Max_depth	9
	Learning_rate	0.1665
	Eval_Metric	logloss
Malware Classification	Min_samples_leaf	(default)
	N_estimators	300
	Random_state	42
	Criterion	gini (default)
	Max_depth	None (default)
	Min_samples_split	2 (default)
Spam Detection	Max_features	sqrt (default)
	Kernel	rbf (default),
	C	1.0 (default),
	Gamma	scale (default);
	feature extraction	TF-IDF (max_features = 10,000)
	Text preprocessing	Lowercasing, non-alphabetic character removal, stopword removal, Porter Stemming

3.3.2 IDS Detection Model Analysis

The Intrusion Detection System (IDS) module is designed to identify malicious network traffic using machine learning techniques.[9] As shown in **Figure 3.2**, the model receives network traffic records collected from benchmark intrusion detection datasets such as CIC-IDS2018. These datasets contain both normal and attack traffic, enabling the model to learn different network behaviors.

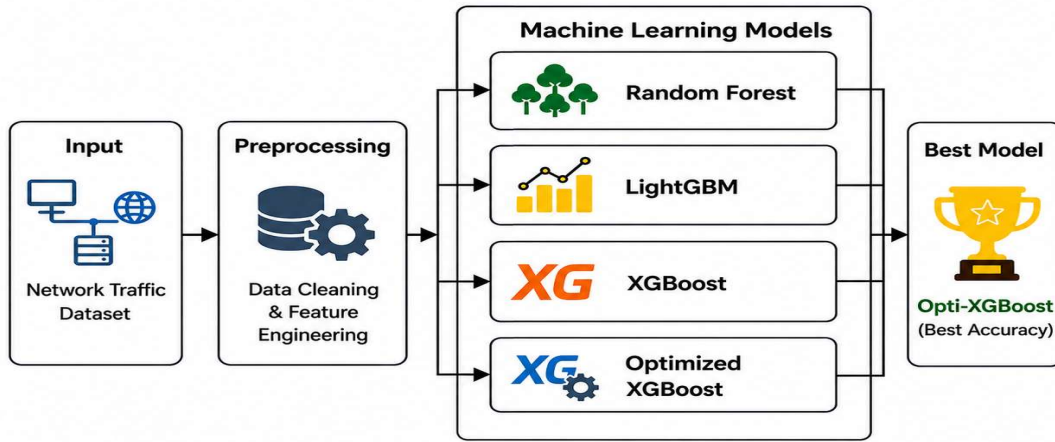


Figure 3.2: Applied ML method for Intrusion Detection System

The collected data is first preprocessed to improve data quality. This stage includes data cleaning, Feature Scaling (StandardScaler), Feature Selection, PCA. The processed dataset is then divided into 80% training data and 20% testing data to evaluate the model on unseen samples. Multiple machine learning algorithms, including Random Forest, LightGBM, XGBoost, Optimized XGBoost are trained to classify network traffic. Each model learns patterns from network features and predicts whether a connection is normal or malicious. The selected model classifies incoming traffic as Normal or Attack.

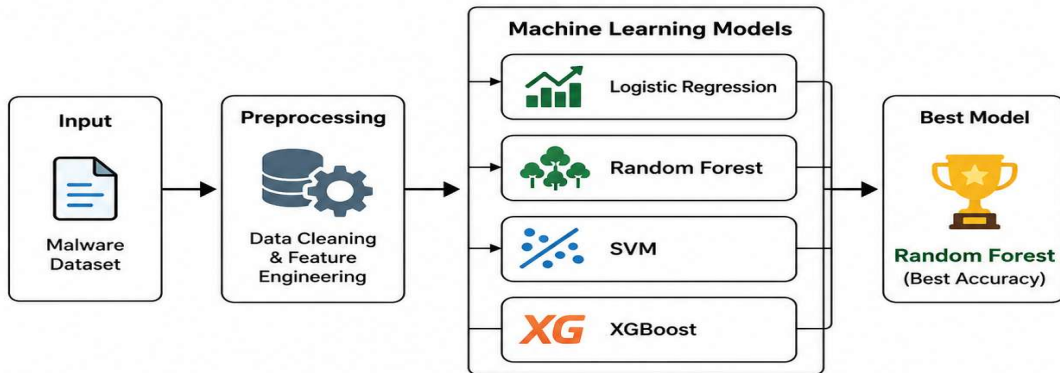


Figure 3.3. Applied ML method for Malware Classification Model

3.3.3 Malware Classification Model Analysis

The Malware Classification module is responsible for identifying malicious executable files using machine learning and deep learning algorithms. As illustrated in **Figure 3.4**, the model analyzes executable files collected from Vectorized byte-file dataset sampled from Microsoft Malware Classification Challenge (BIG 2015). During feature extraction, important characteristics including PE Header information, opcodes, and API calls are extracted from each file. These features describe the internal structure and behavior of executable files,

allowing the model to distinguish between benign and malicious software. The extracted features are divided into 80% training data and 20% testing data. Four classification models— Logistic Regression, Random Forest, SVM, XGBoost —are trained to recognize malware patterns and classify files accurately. After training, the models are evaluated using a train-validation approach. The best-performing model classifies uploaded files as Malware Family Classification

3.3.4 Spam and Phishing Detection Model Analysis

The Spam Detection module identifies phishing and spam emails through Natural Language Processing (NLP) and machine learning techniques. As shown in **Figure 3.5**, the model processes email information including the subject, body text, and Sender collected from the Phishing Email datasets.

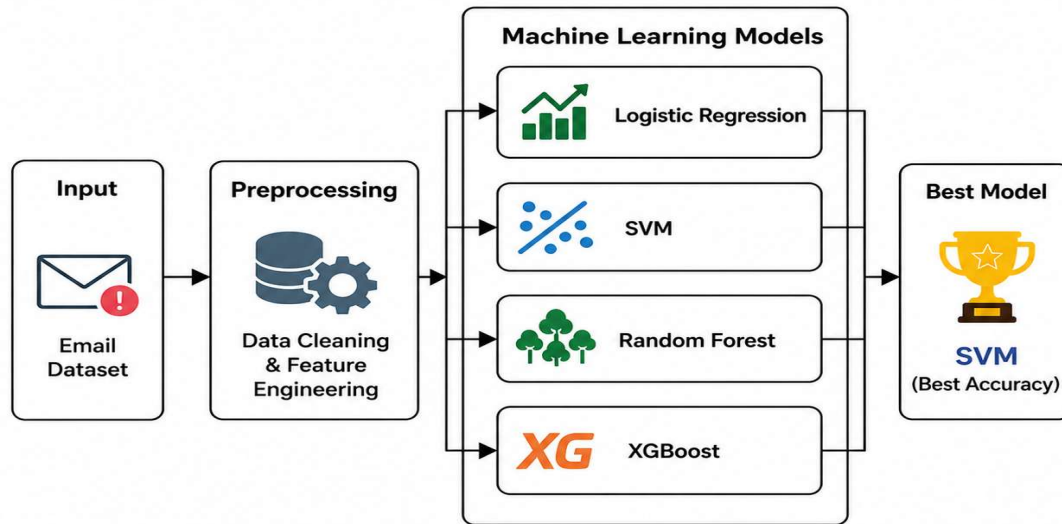


Figure 3.4 Applied ML method of Phishing Detection Model

The email text is first preprocessed using NLP techniques such as tokenization, stop-word removal, and TF-IDF vectorization.[10] These steps convert textual information into numerical features suitable for machine learning algorithms. The processed dataset is divided into 80% training data and 20% testing data. Four classification algorithms Support Vector Machine (SVM), Random Forest, Logistic Regression and XGBoost are trained to distinguish legitimate emails from phishing and spam messages.[11] Finally, the models are evaluated using test accuracy [10].

3.3.5 Proposed Model

IDS–Optimized XGBoost:

Network traffic data isn't clean-cut — a DDoS flow and a heavy but normal traffic burst can look similar on several features at once, and it's usually a combination of features, not one single flag, that gives an attack away. That's the kind of pattern boosting is built for: XGBoost

trains trees one after another, and each new tree is trained specifically on what the previous trees got wrong, so it keeps zeroing in on the traffic samples that are genuinely hard to separate instead of just repeating what it already knows. Random Forest and LightGBM don't do that — RF builds all its trees independently and just averages the votes, so once its trees plateau, more trees don't help it get past the hard cases the way boosting does. We ran Optuna for 15 trials to search combinations of estimators, depth, and learning rate rather than guessing, and it landed on 274 estimators, depth 9, and a learning rate of 0.1665. That's also why we didn't just use default XGBoost — the untuned version already scored 95.39%, but the tuned version specifically pushed recall higher (0.9706), meaning fewer real attacks slipped through as false negatives, which matters more here than accuracy alone.

Malware – Random Forest:

The malware dataset is small and uneven — nine families, most with 200 samples, but Simda has only 42 — and the features come from vectorized bytes, PE headers, and opcodes, which is a lot of noisy, high-dimensional input to learn from with so little data per class. That combination is exactly where Random Forest tends to hold up better than the alternatives: each tree only sees a random subset of samples and features, so no single tree can lock onto the specific quirks of that tiny 42-sample class and call it "learned." Averaging across many such trees cancels out that overfitting risk. XGBoost, by contrast, keeps correcting its own errors iteratively, which works great with enough data but on a class this small it starts memorizing outliers instead of generalizing — and that's part of why it came in slightly behind RF here (92.1% vs. 92.7%) despite being the stronger performer on the IDS data. SVM did far worse (65%) because with this many overlapping byte-level features and this little data per class, it struggled to find a clean separating boundary at all. We left Random Forest mostly at its defaults — 300 trees, gini splitting, sqrt of total features per split — because it didn't need heavy tuning to already outperform everything else.

Phishing/Spam – SVM:

Once an email gets converted into a TF-IDF vector, end up with something like 10,000 dimensions, but almost all of them are zero for any given email — only the handful of words that actually appear carry any signal. Tree-based models aren't a natural fit for that: they work by picking one feature at a time to split on, so with the discriminating signal scattered thinly across thousands of sparse word-features, they either need to grow very deep to capture it or end up ignoring a lot of useful information. SVM works differently — it looks for the cleanest possible margin between the two classes across the whole feature space at once, which is exactly the kind of problem it was originally built for with sparse, high-dimensional text data. That's why it edged out XGBoost (99.67% vs. 99.14%) here even though XGBoost had done better on the more structured IDS and malware data — the nature of the data matters more than which algorithm is "generally strongest." We kept the RBF kernel with the default $C=1.0$ and $\gamma=\text{scale}$, and ran the same preprocessing (lowercasing, stopword removal, Porter stemming) before vectorizing, so the model was working with clean, consistent input rather than raw noisy text.

3.4. Explanation of Different Modules

The proposed SmartShield system integrates the Intrusion Detection System (IDS), Malware Classification, and Spam/Phishing Detection modules into a single AI-powered cybersecurity platform. Rather than operating as standalone applications, these modules communicate through a centralized tool that continuously monitors network activities and user interactions. This unified architecture enables administrators to detect and respond to multiple types of cyber threats from one dashboard.

The SmartShield tool is deployed on a monitoring server connected to the organization's computer network. The server continuously captures network traffic, receives uploaded executable files, and monitors incoming email messages. Each type of data is automatically routed to its corresponding AI detection module without interrupting normal network operations. This centralized approach improves efficiency by eliminating the need for multiple independent security applications.

When network packets are captured from connected devices, the IDS module analyzes the traffic to detect suspicious communication patterns such as intrusion attempts, DoS, or DDoS attacks. If a user uploads a software file through the SmartShield interface, the Malware Classification module extracts important features and determines whether the file is safe or malicious. Similarly, incoming email content is forwarded to the Spam Detection module, where Natural Language Processing techniques identify phishing emails, spam messages, or legitimate communications.

The prediction results generated by all three AI models are transmitted to the SmartShield application through an API-based communication layer. The application stores the detection logs in a centralized database and displays real-time alerts, threat statistics, and historical records on an interactive monitoring dashboard. This enables security administrators to monitor the overall security status of the network from a single interface. The centralized architecture reduces response time, simplifies security management, and enhances the organization's ability to detect and mitigate cyber threats in real time.

Chapter 4

4 Implementation and Result Analysis

This chapter presents the implementation process and performance evaluation of the proposed SmartShield. The system integrates three AI-based detection modules, namely Intrusion Detection, Malware Classification, and Spam/Phishing Detection, into a single cybersecurity platform for real-time threat monitoring. The implementation includes environment setup, data preprocessing, model development, and system integration. Each machine learning model is trained and tested using its respective dataset, and its performance is evaluated using standard metrics such as accuracy, precision, recall, and F1-score. Finally, the results are analyzed and compared to demonstrate the effectiveness of the proposed system in detecting different types of cyber threats.

4.1 Environment Setup

Before discussing the hyperparameter settings of each model, it is important to describe the overall development environment used to build the Smart Shield system. All three modules — Intrusion Detection, Malware Classification, and Phishing/Spam Detection — were developed and trained separately using Kaggle Notebooks, which provided a cloud-based environment with sufficient computational resources for training and testing the models. Python 3.x was used as the primary programming language across all modules, along with standard machine learning and data processing libraries. Table 4.1 summarizes the environment setup, including the development platform, dataset used, data processing steps, libraries, and the train-test split ratio applied for each module.

Table 4.1 Environment Setup of the Proposed method

Component	IDS Model	Malware Classification Model	Phishing Detection Model
Development Platform	Kaggle Notebook	Kaggle Notebook	Kaggle Notebook
Dataset	Intrusion Detection System (IDS) Dataset	Malware Classification Dataset	Phishing Email Spam Detection Dataset
Programming Language	Python 3.x	Python 3.x	Python 3.x
Operating Environment	Kaggle Cloud Notebook	Kaggle Cloud Notebook	Kaggle Cloud Notebook
Data Processing	Label Encoding, SMOTE (Class	Data Cleaning, SMOTE (Class Balancing),	Text Cleaning, Stopword

Component	IDS Model	Malware Classification Model	Phishing Detection Model
	Balancing), Feature Scaling (StandardScaler)	Feature Scaling (StandardScaler), Feature Selection (SelectKBest), PCA	Removal, Stemming (PorterStemmer), TF-IDF Vectorization
Machine Learning Libraries	Pandas, NumPy, Scikit-learn, Optuna, Optimized XGBoost	Pandas, NumPy, Scikit-learn, Random Forest	Pandas, NumPy, Scikit-learn, NLTK, SVM
Visualization Libraries	Matplotlib, Seaborn	Matplotlib, Seaborn	Matplotlib, Seaborn
Train-Test Split	80% Training, 20% Testing	80% Training, 20% Testing	80% Training, 20% Testing
Output	Normal / Attack Detection	Malware Family Classification	Phishing Detection

4.2 Model Implementation

The SmartShield system was developed as a modular cybersecurity platform consisting of three independent machine learning components. Each component was designed to solve a specific security problem using a model that achieved the best performance during the experimental phase. Rather than applying a single algorithm to every task, different machine learning models were selected according to the characteristics of each dataset and the experimental results. The final implementation includes a Random Forest classifier for Malware Classification, a Support Vector Machine (SVM) classifier for email spam and phishing detection, and another Optimized XGBoost classifier for the intrusion detection system.

The overall implementation process followed the same workflow for each module. After preprocessing the datasets, relevant features were extracted and transformed into numerical representations suitable for machine learning. The processed data was then divided into training and testing sets using an 80:20 ratio. Multiple machine learning algorithms were evaluated during experimentation, and the model with the best overall performance was selected for deployment in the SmartShield system.

4.2.1 Malware Classification Module

The Malware Classification module was developed using the Random Forest. Several machine learning algorithms were initially evaluated during the experimentation stage, including Decision Tree, Random Forest, Support Vector Machine, Logistic Regression, and XGBoost. Based on the evaluation results, Random Forest achieved the highest classification accuracy while maintaining strong precision, recall, and F1-score. Consequently, it was selected as the final model for Malware Classification. The training dataset consisted of extracted malware

features obtained from publicly available malware datasets. After preprocessing and label encoding, the Random Forest model was trained using the training subset.

Once training was completed, the trained model was evaluated using previously unseen testing samples. After achieving satisfactory performance, the trained model was serialized using the Python pickle library and stored as a reusable model file. This saved model is loaded by the SmartShield backend whenever malware analysis is requested. Before prediction, uploaded malware samples are converted into byte-level feature vectors using the same preprocessing pipeline employed during model training.

4.2.2 Email Spam and Phishing Detection Module

The email security module was implemented using the Support Vector Machine (SVM) classifier.[16] During experimentation, several traditional machine learning models were trained and compared using the phishing email dataset. Although multiple algorithms produced competitive results, SVM achieved the highest overall accuracy and demonstrated excellent classification capability for textual email data represented using TF-IDF features. Therefore, it was selected as the final classifier for this module. Before model training, each email was converted into a TF-IDF feature vector after undergoing several natural languages preprocessing operations, including lowercasing, punctuation removal, stop-word removal, and stemming. These feature vectors were then used to train the SVM classifier to distinguish between legitimate, spam, and phishing emails.

The trained SVM model learns an optimal decision boundary that separates the different email categories within the high-dimensional feature space generated by TF-IDF. After successful training and evaluation, both the trained SVM model and the TF-IDF vectorizer were saved using the pickle library. During deployment, incoming email text follows the same preprocessing pipeline before being transformed into TF-IDF vectors and classified by the trained SVM model.[17]

4.2.3 Intrusion Detection Module

The intrusion detection component was implemented using the Optimized XGBoost classifier. Similar to the Malware Classification module, different machine learning algorithms were evaluated before selecting the final model. Experimental results showed that Optimized XGBoost consistently outperformed the alternative algorithms in detecting both normal and malicious network traffic while maintaining high classification accuracy and low false positive rates. The network traffic dataset was first preprocessed by removing duplicate records, encoding categorical attributes, and preparing the features for model training. The processed network data was then divided into training and testing subsets. During training, the Optimized XGBoost classifier learned to identify patterns associated with various attack categories as well as normal network behavior.[18]

After training, the model was evaluated using unseen network traffic samples. The trained Optimized XGBoost model was subsequently saved and integrated into the SmartShield backend, allowing real-time intrusion detection during system operation.

4.2.4 Model Deployment

Following successful training and evaluation, all selected models were exported as serialized .pkl files using the Python pickle library. The backend of SmartShield was developed using the Flask framework, where each detection module loads its corresponding trained model during application startup. When a user submits an email, executable file, or network traffic sample through the SmartShield application, the backend first performs the required preprocessing steps before passing the processed input to the appropriate machine learning model. The prediction generated by the model is then returned to the frontend together with the detected security category, enabling users to receive real-time cybersecurity analysis. The modular implementation approach provides flexibility for future improvements. Since each detection module operates independently, individual models can be retrained, replaced, or upgraded without affecting the overall functionality of the SmartShield system.

4.3 Evaluation Metrics

The performance of the machine learning models developed for SmartShield was evaluated using several widely accepted classification metrics. Since the project focuses on cybersecurity tasks such as Malware Classification, spam and phishing email classification, and intrusion detection, relying on a single performance measure is not sufficient. A combination of evaluation metrics was therefore used to provide a comprehensive assessment of each model's classification capability. The evaluation process was carried out using the testing dataset, which consisted of samples that were not used during the training phase. This approach provides a fair estimate of how well the trained models are expected to perform when presented with previously unseen data in real-world situations.

4.3.1 Confusion Matrix

A confusion matrix is one of the most informative tools for evaluating classification models. [11] It summarizes the prediction results by comparing the actual class labels with the predicted class labels. The confusion matrix consists of four fundamental outcomes:

- **True Positive (TP):** Malicious samples correctly identified as malicious.
- **True Negative (TN):** Legitimate samples correctly identified as legitimate.
- **False Positive (FP):** Legitimate samples incorrectly classified as malicious.
- **False Negative (FN):** Malicious samples incorrectly classified as legitimate.

In cybersecurity applications, both false positives and false negatives are important. A high number of false positives may generate unnecessary security alerts, while false negatives are more critical because they allow actual threats to remain undetected.

4.3.2 Accuracy

Accuracy measures the proportion of correctly classified samples among all testing samples. It provides an overall indication of model performance and is one of the most commonly reported evaluation metrics.[11]

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Although accuracy provides a general overview of model performance, it may not fully reflect the effectiveness of a classifier when the dataset is imbalanced. Therefore, additional evaluation metrics were also considered.

4.3.3 Precision

Precision measures how many of the samples predicted as malicious were actually malicious. A high precision value indicates that the model generates fewer false alarms.

$$Precision = \frac{TP}{TP + FP}$$

For cybersecurity systems such as SmartShield, high precision is desirable because it reduces the number of legitimate files, emails, or network activities that are incorrectly flagged as threats.

4.3.4 Recall

Recall, also known as the detection rate or sensitivity, measures the proportion of actual malicious samples that are successfully detected by the model.

$$Recall = \frac{TP}{TP + FN}$$

A high recall value is particularly important in cybersecurity because failing to detect malicious activities may expose the system to significant security risks.

4.3.5 F1-Score

The F1-score combines precision and recall into a single evaluation metric by calculating their harmonic mean. It is especially useful when both false positives and false negatives need to be considered simultaneously.[13]

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

A higher F1-score indicates that the classifier maintains a good balance between correctly identifying threats and minimizing false alarms.

4.3.6 Model Comparison

To determine the most suitable classifier for each SmartShield module, multiple machine learning algorithms were trained and evaluated using the same training and testing datasets. The performance of each model was compared based on Accuracy, Precision, Recall, and F1-score. These metrics provided a balanced evaluation and enabled the selection of the most effective model for deployment.

For the Malware Classification module, Random Forest achieved the best overall performance and was selected as the final model. In the email spam and phishing detection module, the Support Vector Machine (SVM) produced the highest classification accuracy and demonstrated superior performance on TF-IDF feature vectors. Similarly, Optimized XGBoost achieved the best results for the intrusion detection module and was therefore chosen for deployment. The numerical performance results, confusion matrices, and detailed comparisons among all evaluated models are presented in the following section.

4.4 Performance Analysis

The performance of the proposed SmartShield system was evaluated through a series of experiments using publicly available cybersecurity datasets. Each detection module was trained and tested independently using an 80:20 train-test split. Several machine learning algorithms were implemented for every module to identify the most suitable classifier for deployment. The evaluation was carried out using Accuracy, Precision, Recall, F1-score, and Confusion Matrix, as described in the previous section.

The objective of these experiments was not only to identify the model with the highest accuracy but also to evaluate its ability to correctly distinguish malicious activities from legitimate ones while minimizing false predictions. Since each module addresses a different cybersecurity problem, the most appropriate classifier was selected based on its overall performance rather than relying solely on a single evaluation metric.

4.4.1 Malware Classification Performance

To evaluate the Malware Classification module, several supervised machine learning algorithms were trained using the malware dataset. The algorithms included Logistic Regression, Random Forest, Support Vector Machine (SVM), and XGBoost. Each model was trained using the same training dataset and evaluated on the same testing dataset to ensure a fair comparison. **Table 4.2** represents the results comparison for all applied models. **Figure 4.1** illustrates the confusion matrix for malware classification.

Table 4.2. Result comparison for Malware Classification

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.844985	0.853733	0.844985	0.841804
Random Forest	0.927052	0.930059	0.927052	0.926933
SVM	0.653495	0.697759	0.653495	0.627185
XGBoost	0.920973	0.925319	0.920973	0.921446

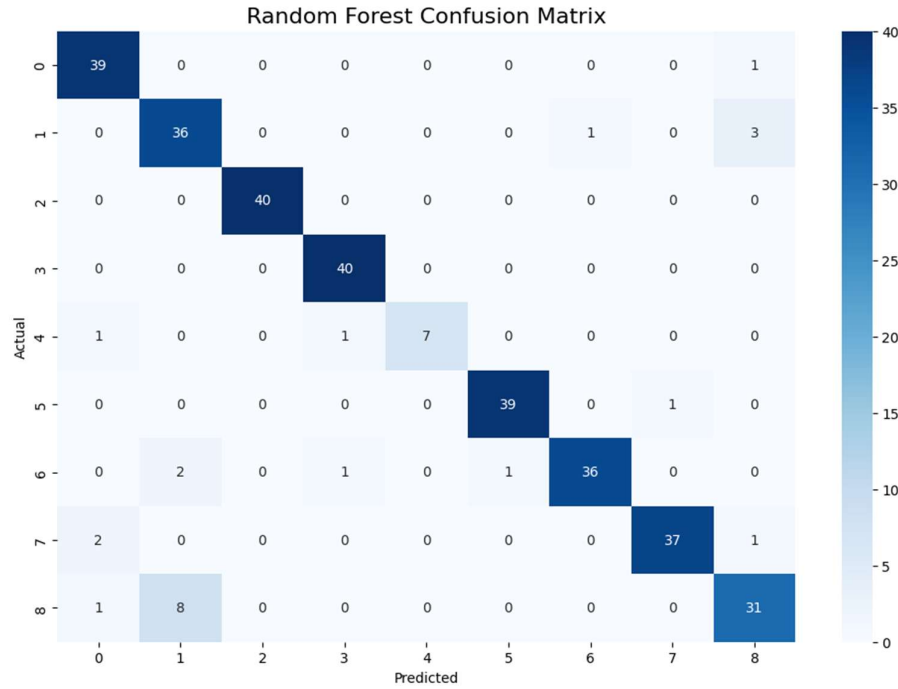


Figure 4.1. Confusion matrix for Malware Detection

The experimental results indicate that the Random Forest classifier consistently outperformed the other machine learning algorithms across all evaluation metrics. The model demonstrated excellent capability in identifying different malware classes while maintaining a low number of misclassifications. The confusion matrix also showed that most malware samples were correctly classified, indicating the effectiveness of the bagging/ensemble approach in handling complex malware feature patterns. Based on these findings, Random Forest was selected as the final Malware Classification model in the SmartShield system.

4.4.2 Email Spam and Phishing Detection Performance

The email security module was evaluated using multiple machine learning classifiers after converting the email texts into TF-IDF feature vectors. Logistic Regression, Random Forest, XGBoost, and Support Vector Machine (SVM) were trained and compared using the same preprocessing pipeline and testing dataset. **Table 4.3** represents the results comparison for all applied models. **Figure 4.2** illustrates the confusion matrix for spam detection.

Table 4.3 Result comparison for Spam and Phishing Detection

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.993871	0.993871	0.993871	0.993870
SVM	0.996680	0.996686	0.996680	0.996680
Random Forest	0.992338	0.992338	0.992338	0.992338
XGBoost	0.991444	0.991459	0.991444	0.991441

The experimental evaluation showed that the Support Vector Machine achieved the highest overall performance for email classification. Unlike the other classifiers, SVM effectively handled the high-dimensional sparse feature vectors generated by the TF-IDF representation. The confusion matrix demonstrated that the model correctly classified the majority of legitimate, spam, and phishing emails while producing comparatively fewer classification errors. As a result, SVM was selected as the final model for the SmartShield email security module.

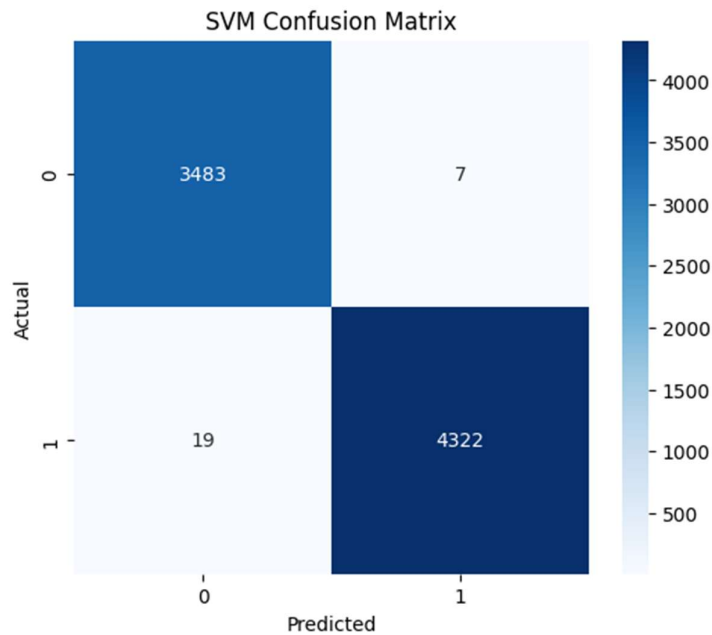


Figure 4.2 Confusion matrix for Phishing Detection

4.4.3 Intrusion Detection System Performance

For the intrusion detection module, multiple machine learning algorithms were trained using the processed network traffic dataset. Each model was evaluated using the same testing dataset to ensure consistent performance comparison. **Table 4.4** represents the results comparison for all applied models. **Figure 4.3** illustrates the confusion matrix for IDS.

Table 4.4. Result comparison for Intrusion Detection

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	0.956714	0.967538	0.968996	0.968806
LightGBM	0.951467	0.970177	0.958243	0.963484
XGBoost	0.953948	0.969020	0.963222	0.966020
Opt-XGB	0.959565	0.970059	0.970628	0.970343

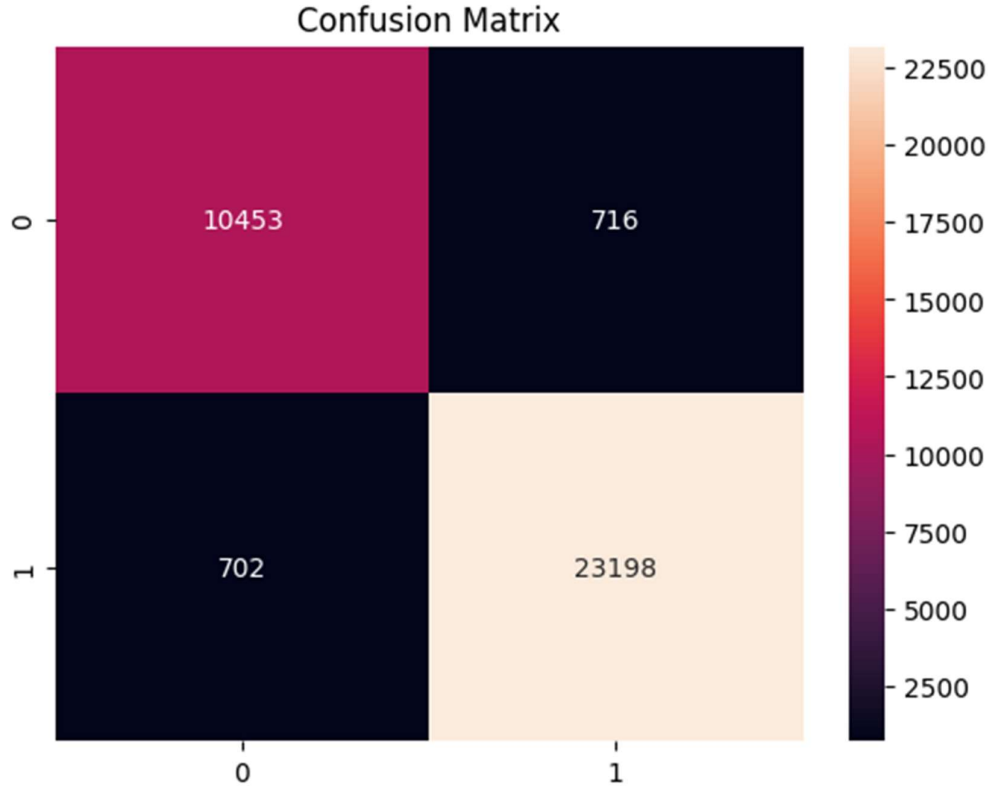


Figure 4.3 Confusion matrix for Intrusion Detection

The Optimized XGBoost Model achieved the highest detection performance among all evaluated models. The model successfully distinguished malicious network traffic from legitimate traffic with high accuracy while maintaining low false positive and false negative rates. The confusion matrix further confirmed that the majority of attack categories were correctly identified, making Optimized XGBoost the most suitable classifier for deployment in the intrusion detection component of SmartShield.

4.4.4 Comparative Analysis

The experimental results demonstrate that no single machine learning algorithm performed best across every cybersecurity task. Instead, the performance of each classifier depended on the characteristics of the dataset and the type of features used for training. For structured datasets containing numerical cybersecurity features, such as malware and network traffic data, Random Forest, Optimized XGBoost consistently achieved the highest performance due to its ensemble learning strategy and ability to model complex relationships between features. On the other hand, the Support Vector Machine proved to be more effective for email spam and phishing detection because of its strong capability to classify high-dimensional sparse text features generated by TF-IDF vectorization.

These findings support the decision to adopt different machine learning models for different SmartShield modules rather than relying on a single classifier for the entire system. This

modular approach allows each component to utilize the most appropriate algorithm, resulting in improved overall system performance and more reliable threat detection.

4.4.5 Deployment Model Selection

Based on the experimental analysis, the final SmartShield implementation uses different classifiers for different security modules. Optimized XGBoost was selected for intrusion detection because it consistently achieved the highest overall performance on structured cybersecurity datasets. For the email security module, the Support Vector Machine was chosen due to its superior classification accuracy and robustness in handling TF-IDF-based text representations.

The selected models were serialized using Python's pickle library and integrated into the Flask backend of the SmartShield application. During system operation, incoming files, emails, and network traffic are preprocessed using the same pipeline employed during training before being classified by their respective machine learning models. This deployment strategy ensures consistency between the training environment and the real-world application.

4.5 Key Findings and Discussions

The experimental results demonstrate that the proposed SmartShield framework can effectively address multiple cybersecurity threats by selecting the most appropriate machine learning model for each security task. Instead of relying on a single classifier, the system adopts a modular architecture in which each module is optimized using the model that achieved the best performance during experimentation.

For the Malware Family Classification module, the Random Forest classifier achieved the highest classification performance. The model effectively learned the patterns within the vectorized malware byte features and accurately classified malware into their respective families. Its ensemble learning approach also reduced misclassification, making it suitable for malware family identification.

For the Spam and Phishing Email Detection module, the Support Vector Machine (SVM) produced the best results. After applying TF-IDF feature extraction, SVM demonstrated excellent performance in handling high-dimensional text data, allowing it to distinguish legitimate emails from spam and phishing emails with high accuracy.

For the Intrusion Detection System (IDS), the Optimized XGBoost model achieved the best overall performance. The model effectively analyzed structured network traffic features and accurately identified malicious activities while maintaining a low false detection rate.

Another important finding is that the choice of machine learning algorithm should depend on the characteristics of the dataset. The experiments showed that no single algorithm performed best for every task. Instead, Random Forest was more effective for malware family classification, SVM was more suitable for text-based email classification, and Optimized XGBoost achieved superior performance for network intrusion detection.

Furthermore, the preprocessing techniques, including data cleaning, feature transformation, feature selection, dimensionality reduction, and class balancing using SMOTE, contributed to improving the overall performance and stability of the models. These findings validate the effectiveness of the proposed SmartShield framework as an intelligent and modular cybersecurity solution capable of supporting malware family classification, spam and phishing email detection, and intrusion detection within a unified platform.

4.6 Deployment

After completing the training and evaluation of the machine learning models, the SmartShield system was deployed as an integrated cybersecurity application capable of performing real-time Malware Classification, spam and phishing email analysis, and intrusion detection. The deployment process focused on creating a modular and scalable environment where each component performs a specific task while communicating through RESTful APIs.

The application backend was developed using the Flask framework, which serves as the central processing unit of the system. The trained Optimized XGBoost, Random Forest and Support Vector Machine (SVM) models were serialized using Python's *pickle* library and loaded into the Flask application during startup. When a prediction request is received, the backend applies the same preprocessing pipeline used during training before passing the processed data to the corresponding machine learning model for inference.

To support real-time intrusion detection, Zeek was integrated into the system to continuously monitor network traffic. Zeek captures packets from the network interface and generates structured network logs containing important connection information. These logs are processed by the backend, where the intrusion detection model analyzes the extracted features and determines whether the observed network activity is normal or malicious.

The frontend communicates with the Flask backend through REST APIs, enabling real-time prediction and result visualization.

Detection history, prediction results, timestamps, and related security information are stored in MongoDB. This allows administrators to review previous detection events, monitor system activity, and maintain a searchable record of cybersecurity incidents without affecting the prediction process.

To simplify deployment and ensure consistent execution across different environments, the entire SmartShield application was containerized using Docker. Separate containers were used for the Flask backend, MongoDB database, and other supporting services. Docker Compose was used to manage these containers, making the system easier to deploy, maintain, and scale. This containerized architecture also reduces dependency conflicts and ensures that the application behaves consistently during both development and deployment.

Figure 4.4 to Figure 4.7 illustrates the deployed SmartShield application. The figures demonstrate the implemented user interface, Malware Classification module, spam and

phishing detection module, intrusion detection dashboard, real-time monitoring interface, and the backend services running inside Docker containers.

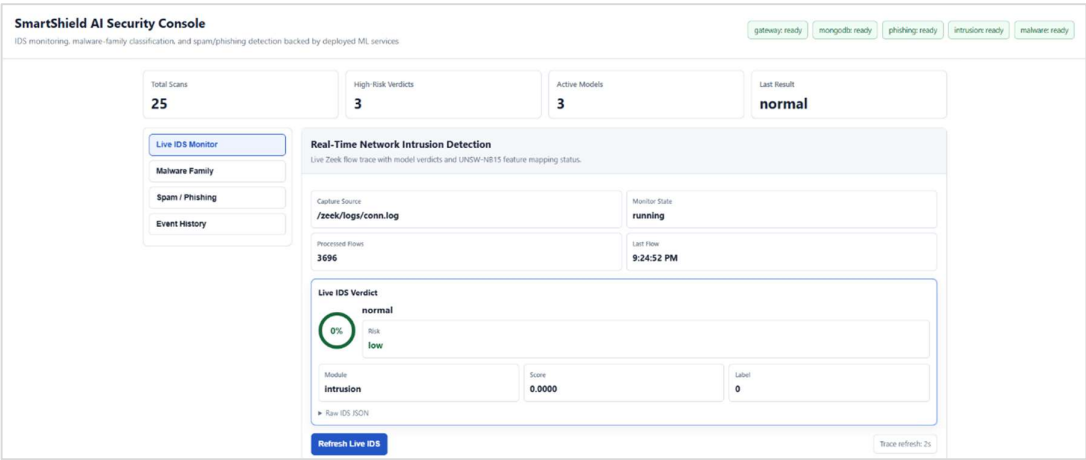


Figure 4.4 Dashboard of the Intrusion Detection module

Figure 4.4 illustrates the Intrusion Detection dashboard of the SmartShield system. The dashboard continuously monitors incoming network traffic and displays whether the captured activities are classified as **Normal** or **Attack** by the trained Optimized XGBoost model. Real-time detection enables administrators to identify suspicious network behavior quickly and respond before potential security incidents escalate.

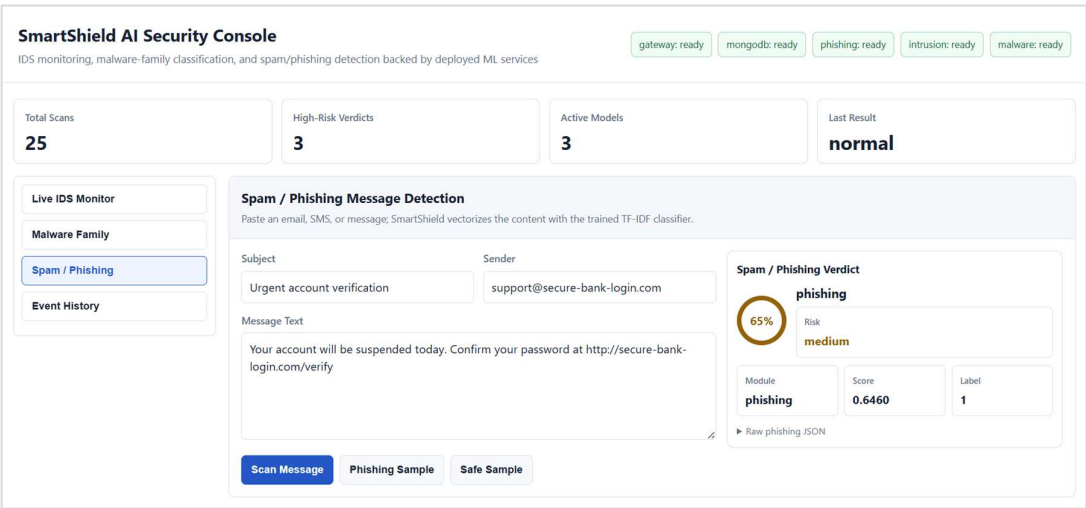


Figure 4.5 Dashboard of Phishing Detection module

Figure 4.5 presents the Email Phishing Detection dashboard integrated into SmartShield. Users can submit email content through the interface, where the trained Support Vector Machine (SVM) model analyzes the text using the same preprocessing pipeline applied during training.

The system instantly classifies the email as **Safe**, **Spam**, or **Phishing**, helping users recognize malicious emails before interacting with them.

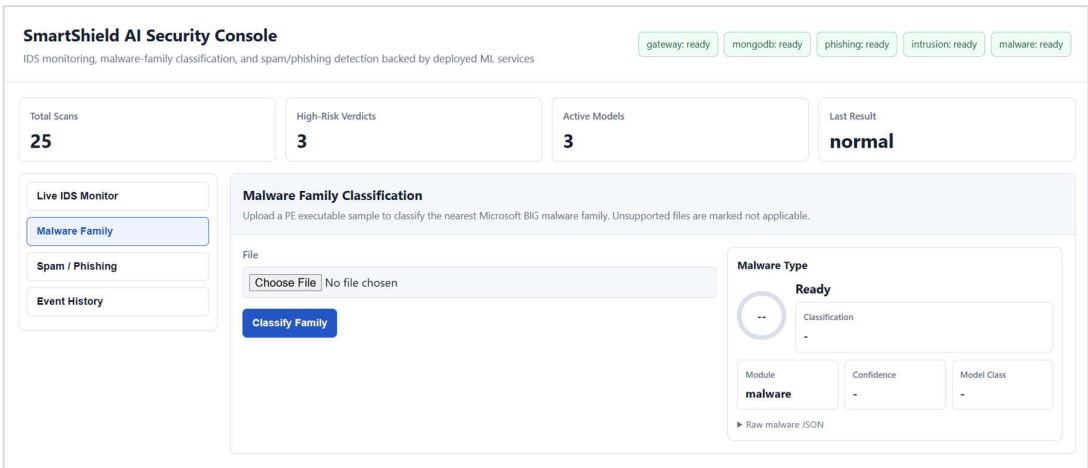


Figure 4.6 Dashboard of the Malware Classification module

Figure 4.6 shows the Malware Classification dashboard, where users can upload executable files for security analysis. The uploaded file is processed using the trained Random Forest model, which identifies the corresponding malware family or determines whether the file is safe. The dashboard provides an easy-to-use interface that enables fast and reliable malware analysis without requiring advanced technical knowledge.

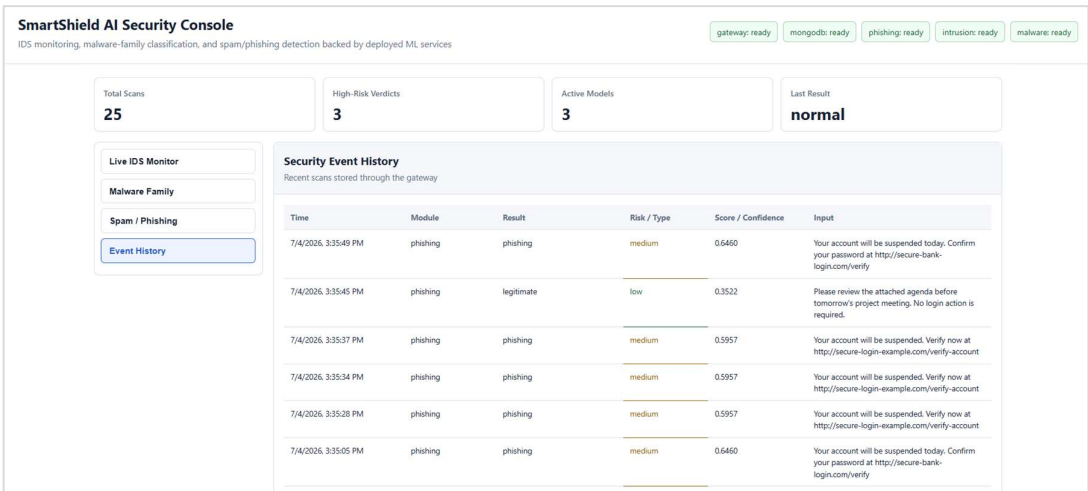


Figure 4.7 Dashboard of the Event history

Figure 4.7 displays Dashboard of the Event history of the SmartShield application. The dashboard maintains a centralized log of all detection events generated by the IDS, Malware Classification, and Phishing Detection modules. It records prediction results together with

timestamps, allowing administrators to review historical incidents, monitor system activity, and support faster investigation and response to potential cyber threats.

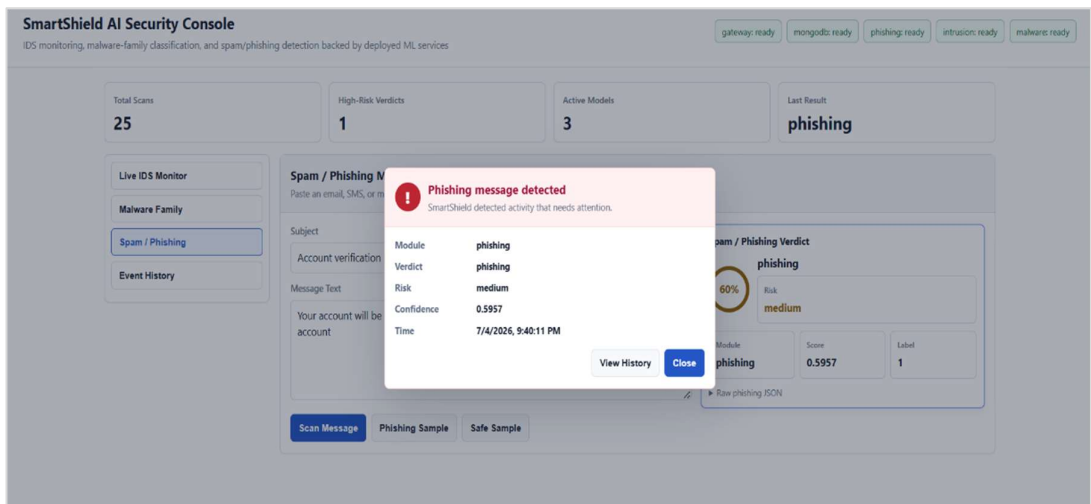


Figure 4.8 Security alert in dashboard

Figure 4.8 shows the Security Alert section of the Smart Shield web dashboard, which displays real-time notifications whenever a threat is detected by any of the three modules. Whenever the Intrusion Detection module flags a malicious packet, the Malware Classification module identifies an infected file, or the Phishing Detection module marks an email as spam or phishing, an alert is instantly generated and shown on this section of the dashboard.

Chapter 5

5 Standards, Constraints, and Milestones

The development of SmartShield involved more than implementing machine learning models. It also required consideration of software engineering standards, ethical responsibilities, project limitations, and long-term sustainability. Since the system is designed to assist users in identifying malware, phishing emails, and network intrusions, its reliability, security, and maintainability are essential for practical deployment.

Throughout the project, established software development practices were followed to ensure that the system remained modular, scalable, and easy to maintain. Appropriate machine learning techniques were selected based on experimental evaluation rather than assumptions, and publicly available datasets were used to train and validate each detection module. In addition, attention was given to ethical issues, data privacy, and the potential societal impact of deploying AI-assisted cybersecurity solutions.

This chapter discusses the sustainability aspects of the proposed system, its potential impact on society, ethical considerations, implementation challenges, project constraints, and the overall development timeline.

5.1. Sustainability Standards

Sustainability is an important consideration in the development of modern software systems.[20] Although SmartShield is a cybersecurity application rather than an environmental project, its design contributes to digital sustainability by improving the efficiency and reliability of cyber threat detection.

The system was designed using a modular architecture in which each security module operates independently. This design allows individual machine learning models to be updated or replaced without affecting the remaining components of the system. Such flexibility reduces future development effort and extends the overall lifespan of the software.

The use of machine learning also contributes to operational efficiency by automating tasks that would otherwise require significant manual effort from cybersecurity professionals. Automated malware analysis, phishing detection, and intrusion detection help reduce response time and improve the consistency of security monitoring.

Open-source technologies such as Python, Flask, and publicly available machine learning libraries were used throughout the project. These technologies minimize software licensing costs while encouraging future research and continuous improvement by the developer community.

From a technical perspective, the selected machine learning models were chosen not only for their detection performance but also for their computational efficiency. Support Vector Machine, Random Forest and Optimized XGBoost provide strong classification performance while remaining practical for deployment on standard computing hardware, making the proposed solution suitable for both educational and small-scale organizational environments.[16][18][19]

5.2. Impacts on Society

Cybersecurity threats continue to increase as more personal, financial, and organizational activities move to digital platforms. Malware attacks, phishing emails, and network intrusions can cause significant financial losses, compromise sensitive information, and disrupt critical services. SmartShield aims to reduce these risks by providing an intelligent platform capable of identifying multiple categories of cyber threats.

For individual users, the system can help identify suspicious emails before they are opened and detect potentially harmful files before execution. This reduces the likelihood of users becoming victims of phishing attacks or malware infections.

Organizations can also benefit from the system by integrating AI-assisted threat detection into their existing security infrastructure. Automated analysis allows security teams to identify potential threats more quickly, improving incident response while reducing the workload associated with manual monitoring.

In educational environments, SmartShield can serve as a practical learning platform for students studying cybersecurity, machine learning, and software engineering. The integration of multiple AI models within a single application demonstrates how artificial intelligence can be applied to solve real-world cybersecurity challenges.

Although SmartShield is not intended to replace professional security analysts, it can function as an intelligent decision-support tool that assists users in identifying potential threats more efficiently.

5.3. Ethics

Ethical considerations play an important role in the development of AI-based cybersecurity systems. Since SmartShield makes automated security decisions, it is important that these decisions are based on reliable machine learning models trained using publicly available and ethically sourced datasets.

Throughout this project, only publicly available datasets intended for research purposes were used. No private user information or confidential organizational data was collected during system development or model training.

The system was designed to assist users rather than make irreversible security decisions automatically. Users remain responsible for verifying security alerts before taking critical

actions. This human-in-the-loop approach helps reduce the risks associated with incorrect predictions or false alarms.

The models were evaluated using multiple performance metrics to minimize false positives and false negatives. Nevertheless, no machine learning model can guarantee perfect accuracy. Therefore, SmartShield should be considered as a decision-support system rather than a complete replacement for conventional cybersecurity practices.

Transparency was also considered during implementation. The project clearly documents the datasets used, preprocessing methods, selected algorithms, and evaluation procedures, allowing future researchers to reproduce and improve the proposed system.

5.4. Challenges

Several technical and practical challenges were encountered during the development of SmartShield. One of the primary challenges was collecting suitable datasets for the three different cybersecurity modules. Each dataset required different preprocessing techniques, making it necessary to design separate data preparation pipelines for Malware Classification, Phishing Detection, and intrusion detection. Another challenge involved selecting the most appropriate machine learning algorithm for each module. Multiple models were trained and evaluated before identifying the algorithms that provided the best balance between classification accuracy and computational efficiency. Training machine learning models on large cybersecurity datasets also required considerable computational resources. Certain algorithms, particularly Support Vector Machine with non-linear kernels, required significant processing time during experimentation. Several optimization techniques were applied to reduce training time while maintaining satisfactory performance.

Integrating multiple AI models into a single application presented additional software engineering challenges. Each module required different preprocessing steps, input formats, and prediction workflows while still maintaining a consistent user experience within the SmartShield platform. Finally, ensuring compatibility between the trained machine learning models and the deployment environment required careful implementation of model serialization, backend API development, and frontend integration.

5.5. Constraints

The development of SmartShield was subject to several practical constraints that influenced both the implementation process and the final system design.

5.5.1 Design Constraints

The system was designed as a modular application to simplify future maintenance and upgrades. Although this architecture improves scalability, it also requires separate preprocessing pipelines and independent machine learning models for each security module.

Maintaining consistency among these components increased the overall implementation complexity.

5.5.2 Component Constraints

The project relies on publicly available datasets, which may not fully represent newly emerging cyber threats. As cyberattack techniques continue to evolve, the trained models require periodic retraining using updated datasets to maintain their effectiveness.

The detection models are also limited by the quality of the extracted features. If important information is absent from the dataset, the prediction accuracy may decrease.

5.5.3. Budget Constraints

SmartShield was developed using open-source software and publicly available machine learning libraries, allowing the project to be completed with minimal financial cost. Development was performed using personal computing resources and cloud-based platforms such as Kaggle for model training. Consequently, no additional expenditure was required for commercial software licenses or specialized hardware.

5.6. Timeline and Gantt Chart

The development of Smart Shield followed a structured timeline spanning from July 2025 to June 2026, divided into eight major phases, as illustrated in Figure X.X. The project began with a Literature Review (July–August 2025), during which existing research on intrusion detection, malware classification, and phishing detection was studied to identify suitable models and techniques. This was followed by Dataset Collection (August–September 2025), where the CICIDS2018, Microsoft Malware Sample, and phishing/spam email datasets were gathered from their respective sources.

Once the datasets were collected, Data Preprocessing (September–October 2025) was carried out, including data cleaning, label encoding, feature scaling, and text preprocessing for each module. This was followed by the Model Training and Evaluation phase (October–December 2025), where multiple machine learning models were trained and compared for each module, and the best-performing model was selected based on accuracy and other evaluation metrics. After finalizing the models, Backend Development (December 2025–February 2026) was carried out to build the core system logic, including packet capturing for the IDS module, file-to-vector conversion for the malware module, and email preprocessing for the phishing module. This was followed by Frontend Integration (February–March 2026), during which the web dashboard was connected with the backend to display real-time detection results.

The system then went through a System Testing phase (March–April 2026) to evaluate its overall performance and reliability. Finally, the project concluded with Documentation and Thesis Writing (April–June 2026), during which the final report was prepared based on the completed system and its results.

Figure 5.1: Project Implementation Timeline (July 2025 – June 2026). Illustrates the overall project timeline and the sequence of activities carried out during the development of the SmartShield system. The Gantt chart presents the major project phases, including literature review, dataset collection, data preprocessing, model development, system implementation, testing, deployment, and report preparation. It provides a clear visualization of task scheduling and progress throughout the project. The timeline also demonstrates how individual activities were planned and completed systematically to ensure the successful development and timely completion of the proposed AI-driven cybersecurity framework.

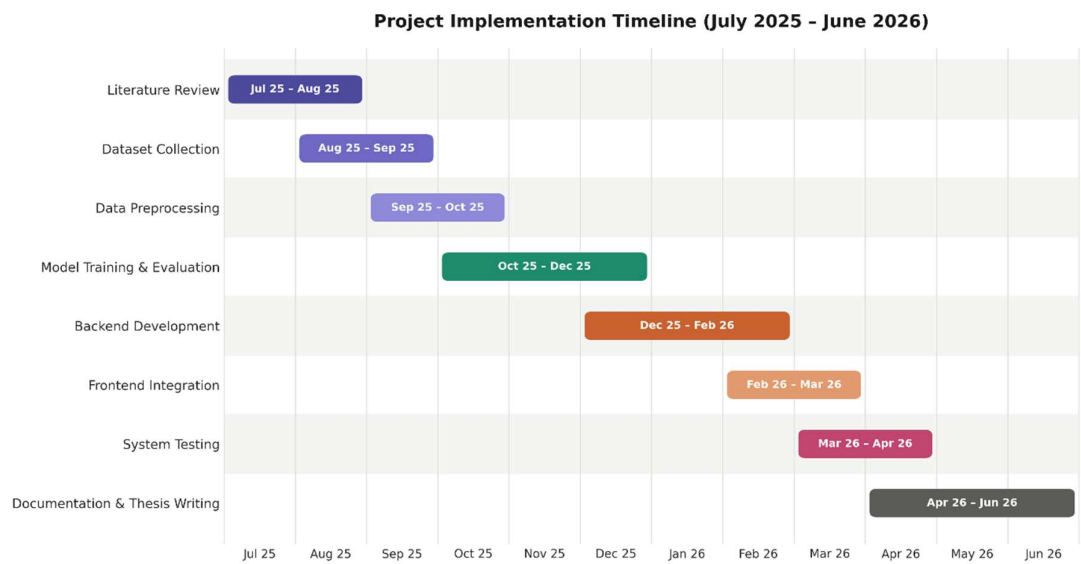


Figure 5.1 Timeline and Gantt Chart

Chapter 6

6 Conclusion

The rapid growth of cyber threats has made intelligent security systems increasingly important for protecting digital assets and online services. Traditional security solutions often rely on predefined rules and signatures, making it difficult to detect newly emerging or evolving attacks. To address this challenge, this project proposed SmartShield, an AI-based cybersecurity framework that integrates Malware Classification, email spam and phishing detection, and intrusion detection into a single intelligent platform.

The development of SmartShield involved the complete machine learning workflow, including dataset collection, data preprocessing, feature extraction, model training, evaluation, and deployment. Multiple machine learning algorithms were experimentally evaluated for each security module to identify the most suitable classifier. Instead of adopting a single algorithm for every task, the final implementation selected different models according to their experimental performance. Random Forest was chosen for the Malware Classification and Optimized XGBoost for intrusion detection modules due to its superior performance on structured cybersecurity datasets, while Support Vector Machine (SVM) demonstrated the best results for spam and phishing email detection after TF-IDF feature extraction.

The trained models were integrated into a Flask-based backend and designed to support real-time prediction through a modular architecture. This design allows each security module to operate independently while maintaining a unified platform for cyber threat analysis. The modular implementation also simplifies future maintenance because individual models can be updated or replaced without affecting the remaining components of the system. Experimental results demonstrated that the proposed approach achieved reliable detection performance across all three cybersecurity tasks. The evaluation using Accuracy, Precision, Recall, F1-score, and Confusion Matrix confirmed the effectiveness of the selected models. The findings indicate that selecting machine learning algorithms based on the characteristics of each dataset leads to better overall system performance than relying on a single universal classifier.

6.1. Limitations

Although SmartShield achieved promising experimental results, several limitations should be acknowledged. The performance of the machine learning models depends heavily on the quality and diversity of the datasets used during training. Publicly available datasets may not fully represent the latest cyber threats or zero-day attacks, which could affect the system's ability to generalize to previously unseen attack patterns.

The current implementation focuses on static malware classification rather than dynamic behavioral analysis. As a result, malware that changes its behavior during execution may require additional analysis techniques beyond those implemented in this project.

Similarly, the email security module primarily analyzes the textual content of emails. Other potentially useful indicators, such as sender reputation, embedded URLs, email headers, attachments, and domain authentication records, were not included in the current implementation. Incorporating these features could further improve phishing detection performance. The intrusion detection module was trained using offline datasets rather than continuous live network traffic. Although the experimental results were encouraging, additional validation using real-world enterprise network environments would provide a more comprehensive assessment of system performance.

Another limitation is that the selected machine learning models require periodic retraining as cybersecurity threats continue to evolve. Without updated training data, prediction accuracy may gradually decrease over time.

6.2 Future Works and Direction

The SmartShield framework provides a strong foundation for future research and further system development. Several enhancements can be considered to improve both the functionality and detection capability of the proposed system. One possible extension is the adoption of advanced deep learning models such as Long Short-Term Memory (LSTM), Bidirectional Encoder Representations from Transformers (BERT), or Transformer-based architectures for email phishing detection. These models are capable of capturing richer contextual information and may improve classification performance on complex phishing emails. For Malware Classification, future work may incorporate dynamic malware analysis by monitoring program behavior during execution. Combining static and dynamic analysis techniques could improve the detection of sophisticated malware variants and zero-day attacks. The intrusion detection module could also be enhanced by integrating live network traffic monitoring, enabling real-time threat detection within operational network environments. This would allow SmartShield to function as a continuous monitoring system rather than relying solely on offline datasets. Another promising research direction involves incorporating additional cybersecurity features into the email analysis module, including URL reputation analysis, domain verification, attachment scanning, email header analysis, and sender authentication mechanisms. Combining these features with machine learning predictions could significantly strengthen phishing detection accuracy. From a deployment perspective, future versions of SmartShield could be implemented as a cloud-based cybersecurity service capable of protecting multiple users and organizations simultaneously. Integration with Security Information and Event Management (SIEM) platforms, cloud security services, and enterprise monitoring tools would further expand its practical applicability.

References

- [1] W. Stallings, *Network Security Essentials: Applications and Standards*, 7th ed. Boston, MA, USA: Pearson, 2017.
- [2] M. Bishop, *Computer Security: Art and Science*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.
- [3] Z. Xu, Y. Wu, S. Wang, J. Gao, T. Qiu, Z. Wang, H. Wan, and X. Zhao, "Deep learning-based intrusion detection systems: A survey," *arXiv preprint arXiv:2504.07839*, 2025, doi: 10.48550/arXiv.2504.07839.
- [4] T. Quertier, B. Marais, G. Barrué, S. Morucci, S. Azé, and S. Salladin, "A lean transformer model for dynamic malware analysis and detection," *arXiv preprint arXiv:2408.02313*, 2024, doi: 10.48550/arXiv.2408.02313.
- [5] H. Bibi, R. Syed, M. Shah, and M. I. Sharif, "Phishing website detection using improved multilayered convolutional neural networks," *J. Comput. Sci.*, vol. 20, no. 9, pp. 1069–1079, 2024, doi: 10.3844/jcssp.2024.1069.1079.
- [6] J. Ł. Wilk-Jakubowski, Ł. Pawlik, G. Wilk-Jakubowski, and A. Sikora, "Machine learning and neural networks for phishing detection: A systematic review (2017–2024)," *Electronics*, vol. 14, no. 18, art. no. 3744, 2025, doi: 10.3390/electronics14183744.
- [7] S. M. Kasongo and Y. Sun, "A deep learning method with filter-based feature engineering for wireless intrusion detection system," *IEEE Access*, vol. 7, pp. 38597–38607, 2019, doi: 10.1109/ACCESS.2019.2905633.
- [8] S. Berman, A. Buczak, J. Chavis, and C. Corbett, "A survey of deep learning methods for cyber security," *Information*, vol. 10, no. 4, art. no. 122, 2019, doi: 10.3390/info10040122.
- [9] M. A. Ferrag, L. Maglaras, S. Moschogiannis, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *J. Inf. Secur. Appl.*, vol. 50, art. no. 102419, 2020, doi: 10.1016/j.jisa.2019.102419.
- [10] solarmainframe, "IDS 2018 intrusion CSVs (CSE-CIC-IDS2018)," Kaggle, 2019. [Online]. Available: <https://www.kaggle.com/datasets/solarmainframe/ids-intrusion-csv>. Accessed: Aug. 2025.
- [11] D. Bhat, "Microsoft malware sample vectorized dataset," Kaggle, 2020. [Online]. Available: <https://www.kaggle.com/datasets/...>. Accessed: Aug. 2025.
- [12] A. Al-Subaiey, M. Al-Thani, N. A. Alam, K. F. Antora, A. Khandakar, and S. A. U. Zaman, "Novel interpretable and robust web-based AI platform for phishing email detection," *arXiv preprint arXiv:2405.11619*, 2024, doi: 10.48550/arXiv.2405.11619.
- [13] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *J. Mach. Learn. Res.*, vol. 3, pp. 1157–1182, 2003, doi: 10.1162/153244303322753616.
- [14] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631, doi: 10.1145/3292500.3330701.
- [15] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995, doi: 10.1007/BF00994018.
- [16] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988, doi: 10.1016/0306-4573(88)90021-0.
- [17] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
- [18] ISO/IEC 25010:2011, *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*. Geneva, Switzerland: ISO, 2011.
- [19] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.